MVC Architecture (Simple Explanation)

MVC (Model–View–Controller) is a way to organize code when building web applications. It divides the application into three parts: **Model**, **View**, and **Controller**. This separation makes the application easier to understand, maintain, and update.

In traditional web development, a single **HTML file** may contain many things together, such as:

- User interface (UI)
- Database operations
- Business logic

When everything is mixed in one file, the code becomes difficult to manage. **MVC solves this problem by separating these responsibilities into different parts.**

1. Model (M)

The **Model** represents the **data of the application**.

- Fields or properties related to the database
- Data structures or classes
- Logic to read, update, or store data in the database

Example: Information about a **user, product, or order**.

2. View (V)

The **View** is the **user interface** of the application. It is responsible for showing data to the user.

- HTML
- CSS
- Page layout and design

Example: A webpage that shows a list of products or a login form.

3. Controller (C)

The **Controller** handles **user actions and application logic**. It acts as a connection between the **Model** and the **View**.

- Receiving user requests
- Processing the request
- Getting data from the Model
- Sending the data to the View to display

Fields Details:

Field Name	Types	Requirement	Details
Student Name	Text	Yes	Name of the student
Class Studying	Dropdown	Yes	Class of the student (e.g., Grade 1–12)
Section	Dropdown	Yes	Section of the class (A/B/C etc.)
Class Teacher Name	Text	Yes	Name of the student's class teacher
Admission Number	Text/Number	Yes	Unique admission number of the student
Registration Number	Text/Number	Yes	Student registration number
Father Name	Text	Yes	Name of the father
Father Occupation	Text	No	Father's occupation
Mother Name	Text	Yes	Name of the mother
Mother Occupation	Text	No	Mother's occupation
Grievance Department	Dropdown	Yes	Department related to complaint (Academic / Transport / Fee / Discipline etc.)
Details of Grievance	Text area	Yes	Full description of the issue
Does the Class Teacher Know?	Radio Button (Yes/No)	Yes	Whether the issue is already known to the class teacher
Parent Email/Student Email	Email	Yes	Email to send confirmation

Validation Step1

1. Student Name

- **Input Type:** Text Box
- **Required:** Yes

Validation Rules:

- Only alphabets and spaces allowed
- Min: 3 characters
- Max: 100 characters

Error Messages:

- Empty → *"Student name is required."*
- Invalid characters → *"Only letters are allowed."*
- Too short → *"Student name must be at least 4 characters."*

2. Class Studying

- **Input Type:** Dropdown

Options Example:

- LKG, UKG, 1, 2, 3 … 12
- **Required:** Yes

Error Messages:

- Not selected → *"Please select class."*

3. Section

- **Input Type:** Dropdown

Options:

- A, B, C, D (configurable)
- **Required:** Yes

Error Messages:

- Not selected → *"Please select section."*

4. Class Teacher Name

- **Input Type:** Text Box
- **Required:** Yes

Validation:

- Alphabets + spaces only
- Min:4 characters
- Max: 100 characters

Error Messages:

- Empty → *"Class teacher name is required."*
- Invalid → *"Only letters are allowed."*

5. Father Name

- **Input Type:** Text Box
- **Required:** Yes

Validation:

- Alphabets + spaces only
- Min:4 characters
- Max: 100 characters

Error Messages:

- Empty → *"Father name is required."*
- Invalid → *"Only letters are allowed."*

6. Father Occupation

- **Input Type:** Text Box
- **Required:** No

Validation:

- Alphabets + spaces only
- Max: 20 characters

Error Message:

- Too long → *"Maximum 100 characters allowed."*

7. Mother Name

- **Input Type:** Text Box
- **Required:** Yes

Validation:

- Alphabets + spaces only
- Min:4 characters
- Max: 100 characters

Error Messages:

- Empty → *"Mother name is required."*
- Invalid → *"Only letters are allowed."*

8. Mother Occupation

- **Input Type:** Text Box
- **Required:** No

Validation:

- Alphabets + spaces only
- Max: 20 characters

Error Message:

- Too long → *"Maximum 100 characters allowed."*

9. Admission Number

- **Input Type:** Text Box
- **Required:** Yes

Validation:

- Alphanumeric only
- No spaces
- Must be unique

Error Messages:

- Empty → *"Admission number is required."*
- Invalid → *"Only letters and numbers allowed."*
- Duplicate → *"Admission number already exists."*

10. Registration Number

- **Input Type:** Text Box
- **Required:** Yes

Validation:

- Alphanumeric
- Must be unique

Error Messages:

- Empty → *"Registration number is required."*
- Duplicate → *"Registration number already exists."*

11. Grievance Department

- **Input Type:** Dropdown

Options:

- Academic
- Transport
- Fees
- Infrastructure
- Others
- **Required:** Yes

If the **user** click the **Others**,
then,

Input Type: Text box

Validation:

- Alphabets + spaces only
- Min:4 characters
- Max: 100 characters

Error Messages:

- Not selected → *"Please select grievance department."*

12. Details of Grievance

- **Input Type:** Text Area (Multi-line)
- **Required:** Yes

Validation:

- Min: 10 characters
- Max: 1000 characters

Error Messages:

- Empty → *"Please enter grievance details."*
- Too short → *"Minimum 10 characters required."*

13. Does Class Teacher Know?

- **Input Type:** Radio Button

Options:

- Yes
- No
- **Required:** Yes

Error Messages:

- Not selected → *"Please select an option."*

After the form submit step 2

User fills form and clicks Submit,

1. User Submits the Form

User fills all fields and clicks Submit

Data exists temporarily in the browser (frontend memory)

At this stage:

- Nothing is saved yet
- It's just user input

Triggers the whole process

User clicks Submit button

In an ASP.NET MVC app:

- The form is tied to a **Controller Action Method**

When user clicks submit:

- Browser sends an **HTTP POST request**
- URL → `/Grievance/Create`

2. Frontend Validation (First Level)

Before sending data:

Each field is checked:

- Empty fields?
- Wrong format?
- Invalid values?

Example:

If Student Name is empty → show error immediately

If mobile number is invalid → block submission Like that,

3. Backend Validation (Second Level – Important)
Data send to the backend

Now backend **re-validates everything**.

Why again?

- Frontend can be bypassed (user can hack/skip it)
- Backend is the **final authority**

Backend checks:

- Required fields present?
- Data format correct?
- Admission number already exists?
- Any invalid or suspicious input?

If anything fails:

- Backend **rejects request**
- Sends error response

How the backend will Validates:

Backend Method Flow:

When request comes:

1. Backend reads incoming data

2. It checks rules defined in backend

Validation happens using:

Condition checks

- Is field empty?
- Is format correct? Is value allowed?

Business rules

- Admission number already exists?
- Required fields present?

If any rule fails:

- Backend stops process
- Sends error response

Validation rules are written in backend methods/classes

If the Validation is successful from the backend side, then its comes to the process of Data Sanitization:

After validation Step 3

1. Data Sanitization (WHERE & HOW)

Happens AFTER validation, BEFORE DB

- Removes spaces
- Can also:
    - Remove special chars
    - Normalize case

2. Passing Data to Data Access Layer

Now controller calls a method:

This is our **Data Access Method** through the **ADO.Net.**

Here, we use the **parameterized query**. So instead of using the direct query(which affect the code, why because direct query- input taken as a code, not a data, that's the problem), that's why we using the parameterized query.

After Access the Data, which value goes into which column Step 4

1. Data Mapping to Query

Data **Mapping** is used to connect application data with database structure**.** Links model properties to DB columns,

- Model property → SQL parameter

How its work,

model.StudentName → @StudentName → StudentName column

2. Inside Database

Database does:

Step-by-step:

1. Parse SQL query
2. Validate table exists
3. Check columns
4. Apply constraints:
    - NOT NULL
    - UNIQUE

3. Row Creation

If everything is valid:

- New row is created
- Values inserted into columns

4. Physical Storage/Cloud Storage

- Data stored in:

  Memory → Disk

- Transaction committed

Now it is **permanently saved**

How its actually work on Database

In application, we need to store and retrieve data from the database.
Instead of writing complex SQL queries for every operation, we use a concept called **ORM (Object-Relational Mapping)**.

ORM acts like a bridge between the application and the database. It allows us to work with data using normal C# classes instead of directly writing database queries.

ORM

ORM helps to connect:

- Application objects (like C# classes)
- Database tables

So whatever data we handle in the application is automatically mapped to the database.

In ASP.NET MVC, we commonly use **Entity Framework Core** as the ORM tool.

To use it, we need to install some required packages:

- Entity Framework Core
- Entity Framework Core Tools
- SQL Server provider

These packages help the application connect to the database and perform operations easily.

Two Ways to Work with Database

There are two main approaches:

Code First Approach

In this approach, we create everything inside the application.

- We define C# classes (models)
- These models represent database tables
- Then the database is automatically created based on these classes

In simple terms:

First we write code, then the database is created from that code.

Database First Approach

In this approach, the database already exists.

- Tables are already created in SQL Server
- The application reads those tables
- Then it generates C# classes based on the database

In simple terms:

First database exists, then the application connects to it.

How Data is Stored (Simple Flow)

When a user submits a form:

1. The data comes to the controller
2. It is stored in a model (C# object)
3. ORM converts this object into a database query
4. The data is saved into the corresponding table

Why We Use ORM

- No need to write SQL for every operation
- Easier to manage data
- Reduces errors
- Makes development faster

Overall flow:

User clicks Submit → Frontend Brower validate → HTTP POST Request sent → Controller Method triggered→Backend Validation (re-val)→ Data Sanitization (Trim) → Call Data Access Method(through Ado.net)→ Prepare Parameterized Query → Map Model → SQL Parameter→ Execute Query→ Database validates & inserts row→ Data stored permanently →Response returned.

Email Notification Process:

Once the data are successfully saved in the databased, then only email is trigger.

STEP 1

1. Backend Triggers Email Logic

After saving data:

- Backend calls an **Email Service / Method**

Once data is saved → its call the email function(service and controller)

STEP 2

2. HOW BACKEND CALLS EMAIL SERVICE

Backend is divided into layers:

```
Controller → Service → Email Service
```

Step-by-step:

1. **Controller receives form**

    - Validates
    - Calls service

2. **Service saves data**

    - Calls DB layer
    - Gets success response

STEP 3

3 . Email Content Preparation

Backend prepares **two different emails**

A. Parent Email (Thank You Mail)

Includes:

- Student Name
- Grievance ID
- Message: “Your grievance has been received”

B. Admin Email

Includes:

- Full form details
- Department
- Description

STEP 4

WHAT EMAIL SERVICE DOES

Email service is responsible for:

1. Preparing email content
2. Connecting to SMTP
3. Sending email

HOW BACKEND CONNECTS TO SMTP SERVER

Required SMTP Details

- SMTP Host (e.g., smtp.gmail.com)
- Port (e.g., 587)
- Username (email ID)
- Password / App password

Connection Process

Step-by-step:

1. Backend creates SMTP client
2. Sets server details
3. Provides credentials
4. Opens connection

Authentication

SMTP server checks:

- Is this sender valid?
- Are credentials correct?

If yes → allow sending
If no → reject

STEP 5

5. EMAIL DELIVERY PROCESS

Once backend sends email to SMTP:

SMTP Server Process

1. Accept email request
2. Identify recipient domain
    - example: gmail.com

3. Contact recipient mail server
4. Transfer email

ADMIN DASHBOARD:

Step 1. Separate Admin Table – How It Is Connected

The system maintains a **separate table for admin users** to ensure:

- Only authorized users can access dashboard, Normal users (students/parents) cannot log in as admin

Backend Validation Logic

1. Check if username exists through the mapping concept
2. Compare password (hashed comparison)

Output

- Success → Redirect to dashboard
- Failure → Show "Invalid credentials.

The given login credential is checked in the admin table, if its success, its redirect to the dashboard.

After successful login:

Backend creates a **session / token for that login id.**

Step 2: Dashboard Page Load

- When Admin opens the dashboard

How it works:

- Browser sends a request to load dashboard page
- This request includes the session ID

Backend now knows which user is accessing.

Step 3: Session Validation

- System checks whether the admin is logged in, then

- Backend verifies session ID
- If session exists and valid → allow access
- If not → redirect to login page

Step 4: Fetch Data from Database

- All grievance data is retrieved

- Controller calls service/repository
- ORM (Entity Framework) generates a query
- Database returns records

Data is converted into objects and sent to the controller.

Step 5: Display Data in Table

- Data is shown in dashboard

How it works:

- Controller sends data to the view (UI)
- UI loops through data and displays it in table format

Step 6: Apply Search

- Admin enters search keyword

How it works:

- Keyword is sent to backend
- Backend modifies query using search condition (LIKE)
- Database returns matching results

Only relevant records are shown.

Step 7: Apply Filters

- Admin selects filters (Class, Section, etc.)

How it works:

- Selected values are sent to backend
- Backend dynamically builds query conditions

Example:

```
WHERE Class = 10 AND Section = 'A'
```

Only filtered data is retrieved from database.

Step 8: Filtered Data Display

- Filtered results are shown

- Backend sends filtered data
- UI refreshes table with new results

No need to reload full data.